

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

RUMAISA SHARIFF (1WA24CS242)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by RUMAISA SHARIFF (1WA24CS242), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026

. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Preemptive and Non-Preemptive) c) Priority (Preemptive and Non-Preemptive) d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-18
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	19-22
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	23-29
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	30-34
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	35-39
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	40-42
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	43-47

LAB PROGRAM 1

a)FCFS

```
#include <stdio.h>

int main()
{
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int bt[50], wt[50], tat[50], at[50], ct[50];
    float total_wt = 0, total_tat = 0;
    for(i = 0; i < n; i++)
    {
        printf("Enter arrival time for process %d: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter burst time for process %d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    ct[0] = at[0] + bt[0];
    wt[0] = 0;
    tat[0] = bt[0];

    for(i = 1; i < n; i++)
    {
        if(ct[i - 1] < at[i])
        {
            ct[i] = at[i] + bt[i];
            wt[i] = 0;
        }
    }
}
```

```

    }
    else
    {
        ct[i] = ct[i - 1] + bt[i];
        wt[i] = ct[i - 1] - at[i];
    }

    tat[i] = bt[i] + wt[i];
}

printf("\nProcess\tAT\tBT\tWT\tTAT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

return 0;

```

}

```
"C:\Users\Admin\Documents\1WA24CS242 OS LAB\FCFS PGM 1a.exe"
Enter number of processes: 4
Enter arrival time for process 1: 0
Enter burst time for process 1: 2
Enter arrival time for process 2: 1
Enter burst time for process 2: 4
Enter arrival time for process 3: 2
Enter burst time for process 3: 6
Enter arrival time for process 4: 3
Enter burst time for process 4: 8

Process AT      BT      WT      TAT
P1      0        2        0        2
P2      1        4        1        5
P3      2        6        4       10
P4      3        8        9       17

Average Waiting Time = 3.500000
Average Turnaround Time = 8.500000

Process returned 0 (0x0)   execution time : 18.427 s
Press any key to continue.
```

b) SJF (Non Preemptive)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j,pos,temp;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    int bt[50],wt[50],tat[50],at[50],ct[50];
```

```
    float total_wt=0,total_tat=0;
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        printf("Enter arrival time for process %d: ",i+1);
```

```
        scanf("%d",&at[i]);
```

```
        printf("Enter burst time for process %d: ",i+1);
```

```
        scanf("%d",&bt[i]);
```

```
    }
```

```
for(i=0;i<n;i++)  
{  
    pos=i;  
    for(j=i+1;j<n;j++)  
    {  
        if(bt[j]<bt[pos])  
            pos=j;  
    }
```

```
    temp=bt[i];  
    bt[i]=bt[pos];  
    bt[pos]=temp;
```

```
    temp=at[i];  
    at[i]=at[pos];  
    at[pos]=temp;  
}
```

```
ct[0]=at[0]+bt[0];  
wt[0]=0;  
tat[0]=bt[0];
```

```
for(i=1;i<n;i++)  
{  
    if(ct[i-1]<at[i])  
    {  
        ct[i]=at[i]+bt[i];  
        wt[i]=0;  
    }
```

```
    else
    {
        ct[i]=ct[i-1]+bt[i];
        wt[i]=ct[i-1]-at[i];
    }

    tat[i]=bt[i]+wt[i];
}

printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],wt[i],tat[i]);
    total_wt+=wt[i];
    total_tat+=tat[i];
}

printf("\nAverage Waiting Time = %.2f",total_wt/n);
printf("\nAverage Turnaround Time = %.2f\n",total_tat/n);

return 0;
```


}

```
"C:\Users\Admin\Documents\1WA24CS242 OS LAB\SJF NON-PREEMPTIVE PGM 1b.exe"
Enter number of processes: 3
Enter arrival time for process 1: 1
Enter burst time for process 1: 4
Enter arrival time for process 2: 0
Enter burst time for process 2: 7
Enter arrival time for process 3: 2
Enter burst time for process 3: 4

Process AT      BT      CT      WT      TAT
P1      1       4       5       0       4
P2      2       4       9       3       7
P3      0       7      16       9      16

Average Waiting Time = 4.000000
Average Turnaround Time = 9.000000

Process returned 0 (0x0)   execution time : 8.821 s
Press any key to continue.
```

c) SJF (Preemptive)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,time=0,smallest,count=0;
```

```
    int bt[50],rt[50],wt[50],tat[50],at[50],ct[50];
```

```
    float total_wt=0,total_tat=0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        printf("Enter arrival time for process %d: ",i+1);
```

```
        scanf("%d",&at[i]);
```

```
        printf("Enter burst time for process %d: ",i+1);
```

```
        scanf("%d",&bt[i]);
```

```
        rt[i]=bt[i];
```

```
}
```

```
while(count!=n)
```

```
{
```

```
    smallest=-1;
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        if(at[i]<=time && rt[i]>0)
```

```
        {
```

```
            if(smallest==-1 || rt[i]<rt[smallest])
```

```
                smallest=i;
```

```
        }
```

```
    }
```

```
    if(smallest!=-1)
```

```
    {
```

```
        time++;
```

```
        continue;
```

```
    }
```

```
    rt[smallest]--;
```

```
    time++;
```

```
    if(rt[smallest]==0)
```

```
    {
```

```
        count++;
```

```
        ct[smallest]=time;
```

```
        tat[smallest]=ct[smallest]-at[smallest];
```

```
        wt[smallest]=tat[smallest]-bt[smallest];
```

```

        total_wt+=wt[smallest];
        total_tat+=tat[smallest];
    }
}

printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],ct[i],wt[i],tat[i]);
}

printf("\nAverage Waiting Time = %.2f",total_wt/n);
printf("\nAverage Turnaround Time = %.2f\n",total_tat/n);

return 0;
}

```

```

"C:\Users\Admin\Documents\1WA24CS242 OS LAB\SJF PREEMPTIVE PGM 1c.exe"
Enter number of processes: 3
Enter arrival time for process 1: 4
Enter burst time for process 1: 3
Enter arrival time for process 2: 5
Enter burst time for process 2: 6
Enter arrival time for process 3: 1
Enter burst time for process 3: 7

Process AT    BT    CT    WT    TAT
P1      4      3      7      0      3
P2      5      6     17      6     12
P3      1      7     11      3     10

Average Waiting Time = 3.000000
Average Turnaround Time = 8.333333

Process returned 0 (0x0)   execution time : 21.996 s
Press any key to continue.

```

d)Priority (Non Preemptive)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int process_id[n], arrival_time[n], burst_time[n], priority[n];
    int completion_time[n], turnaround_time[n], waiting_time[n];
    int completed[n];
    for(int i = 0; i < n; i++) {
        printf("\nEnter Process ID: ");
        scanf("%d", &process_id[i]);
        printf("Enter Arrival Time: ");
        scanf("%d", &arrival_time[i]);
        printf("Enter Burst Time: ");
        scanf("%d", &burst_time[i]);
        printf("Enter Priority: ");
        scanf("%d", &priority[i]);
        completed[i] = 0;
    }
    int current_time = 0;
    int completed_count = 0;
    while(completed_count < n) {
        int highest_priority = 9999;
        int selected_process = -1;
        for(int i = 0; i < n; i++) {
            if(arrival_time[i] <= current_time && completed[i] == 0) {
                if(priority[i] < highest_priority) {
                    highest_priority = priority[i];
                }
            }
        }
        if(selected_process != -1) {
            current_time += burst_time[selected_process];
            completed[selected_process] = 1;
            completed_count++;
        }
    }
}
```

```

selected_process = i;
}
}
}
if(selected_process == -1) {
current_time++;
} else {
int start_time = current_time;
completion_time[selected_process] =
start_time + burst_time[selected_process];
turnaround_time[selected_process] =
completion_time[selected_process] - arrival_time[selected_process];
waiting_time[selected_process] =
turnaround_time[selected_process] - burst_time[selected_process];
current_time = completion_time[selected_process];
completed[selected_process] = 1;
completed_count++;
}
}
printf("\nPID\tAT\tBT\tPR\tWT\tTAT\n");
for(int i = 0; i < n; i++) {
printf("%d\t%d\t%d\t%d\t%d\t%d\n",
process_id[i],
arrival_time[i],
burst_time[i],
priority[i],
waiting_time[i],
turnaround_time[i]);
}
return 0;}

```

```
C:\Users\Admin\Documents\ X + v
Enter number of processes: 3

Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 5
Enter Priority: 3

Enter Process ID: 2
Enter Arrival Time: 4
Enter Burst Time: 8
Enter Priority: 2

Enter Process ID: 3
Enter Arrival Time: 2
Enter Burst Time: 5
Enter Priority: 3

PID    AT    BT    PR    WT    TAT
1       0     5     3     0     5
2       4     8     2     1     9
3       2     5     3    11    16

Process returned 0 (0x0)   execution time : 41.304 s
Press any key to continue.
```

e)Priority (Preemptive)

```
#include <stdio.h>
```

```
int main() {
```

```
int n, rule;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
printf("Enter priorities (1 for higher number means higher priority, 2 for lower number  
means higher priority): ");
```

```
scanf("%d", &rule);
```

```
int pid[n], at[n], bt[n], pr[n];
```

```
int rt[n], ct[n], tat[n], wt[n];
```

```
int completed = 0, current_time = 0;
```

```
printf("Enter process ids:\n");
```

```
for(int i = 0; i < n; i++)
```

```
scanf("%d", &pid[i]);
```

```
printf("Enter arrival times:\n");
```

```
for(int i = 0; i < n; i++)
```

```
scanf("%d", &at[i]);
```

```

printf("Enter burst times:\n");
for(int i = 0; i < n; i++) {
scanf("%d", &bt[i]);
rt[i] = bt[i];
}
printf("Enter priorities:\n");
for(int i = 0; i < n; i++)
scanf("%d", &pr[i]);
while(completed < n) {
int selected = -1;
for(int i = 0; i < n; i++) {
if(at[i] <= current_time && rt[i] > 0) {
if(selected == -1)
selected = i;
else {
if(rule == 1) {
if(pr[i] > pr[selected])
selected = i;
}
else {
if(pr[i] < pr[selected])
selected = i;
}
}
}
}
if(selected == -1) {
current_time++;
}
else {

```

```

rt[selected]--;
current_time++;
if(rt[selected] == 0) {
ct[selected] = current_time;
tat[selected] = ct[selected] - at[selected];
wt[selected] = tat[selected] - bt[selected];
completed++;
}
}

}

float avg_tat = 0, avg_wt = 0;
printf("\nPriority Scheduling (Pre-Emptive)\n\n");
printf("PID\tPrior\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(int i = 0; i < n; i++) {
printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
pid[i], pr[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
avg_tat += tat[i];
avg_wt += wt[i];
}
printf("\nAverage turnaround time: %.2f\n", avg_tat/n);
printf("Average waiting time: %.2f\n", avg_wt/n);
return 0;

```


}

```
"C:\Users\Admin\Documents\" X + v
Enter number of processes: 3
Enter priorities (1 for higher number means higher priority, 2 for lower number means higher priority): 1
Enter process ids:
1 2 3
Enter arrival times:
0 2 5
Enter burst times:
4 6 8
Enter priorities:
2 1 4

Priority Scheduling (Pre-Emptive)

PID  Prior  AT  BT  CT  TAT  WT  RT
1     2     0   4   4   4    0   0
2     1     2   6  18  16   10   0
3     4     5   8  13   8    0   0

Average turnaround time: 9.33
Average waiting time: 3.33

Process returned 0 (0x0)   execution time : 30.772 s
Press any key to continue.
|
```

f) Round Robin

#include <stdio.h>

```
int main() {
```

```
int n, tq;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
printf("Enter Time Quantum: ");
```

```
scanf("%d", &tq);
```

```
int pid[n], at[n], bt[n], rt[n];
```

```
int ct[n], tat[n], wt[n];
```

```
int queue[100], front = 0, rear = 0;
```

```
int visited[n];
```

```
for(int i = 0; i < n; i++) {
```

```
pid[i] = i + 1;
```

```
visited[i] = 0;
```

```
}
```

```
printf("Enter arrival times:\n");
```

```
for(int i = 0; i < n; i++)
```

```
scanf("%d", &at[i]);
```

```
printf("Enter burst times:\n");
```

```
for(int i = 0; i < n; i++) {
```

```
scanf("%d", &bt[i]);
```

```
rt[i] = bt[i];
```

```
}
```

```
int current_time = 0, completed = 0;
```

```
while(completed < n) {
```

```
for(int i = 0; i < n; i++) {
```

```
if(at[i] <= current_time && visited[i] == 0) {
```

```
queue[rear++] = i;
```

```
visited[i] = 1;
```

```
}
```

```
}
```

```
if(front == rear) {
```

```
current_time++;
```

```
continue;
```

```
}
```

```
int p = queue[front++];
```

```
if(rt[p] > tq) {
```

```
current_time += tq;
```

```
rt[p] -= tq;
```

```
for(int i = 0; i < n; i++) {
```

```
if(at[i] <= current_time && visited[i] == 0) {
```

```
queue[rear++] = i;
```

```
visited[i] = 1;
```

```
}
```

```
}
```

```
queue[rear++] = p;
```

```
}
```

```
else {
```

```
current_time += rt[p];
```

```
rt[p] = 0;
```

```
ct[p] = current_time;
```

```
tat[p] = ct[p] - at[p];
```

```
wt[p] = tat[p] - bt[p];
```

```
completed++;
```

```
}
```

```
}
```

```
float avg_tat = 0, avg_wt = 0;
```

```
printf("\nRRS scheduling:\n\n");
```

```
printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
```

```

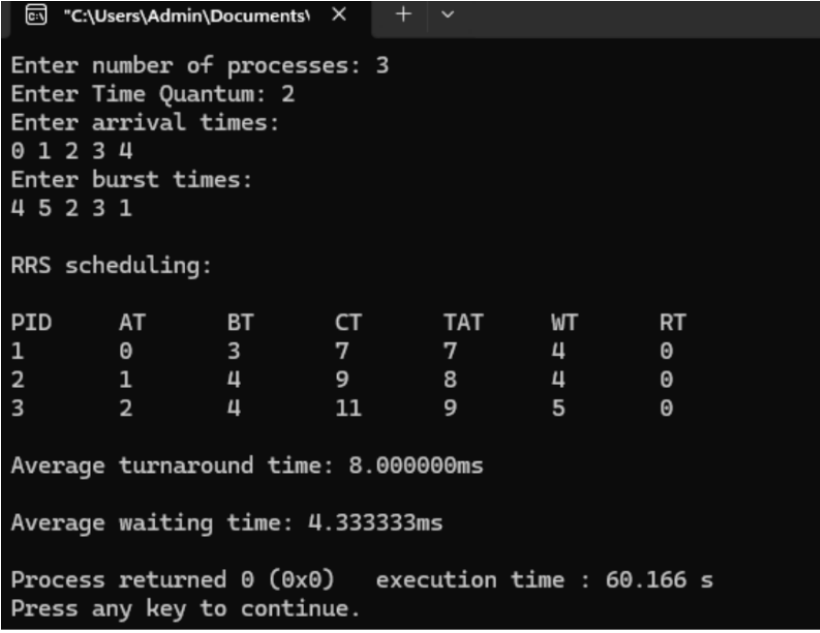
for(int i = 0; i < n; i++) {
printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
pid[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);

avg_tat += tat[i];
avg_wt += wt[i];
}

printf("\nAverage turnaround time: %.6fms\n", avg_tat/n);
printf("\nAverage waiting time: %.6fms\n", avg_wt/n);

return 0;
}

```



```

C:\Users\Admin\Documents\ >
Enter number of processes: 3
Enter Time Quantum: 2
Enter arrival times:
0 1 2 3 4
Enter burst times:
4 5 2 3 1

RRS scheduling:

PID    AT    BT    CT    TAT    WT    RT
1       0     3     7     7     4     0
2       1     4     9     8     4     0
3       2     4    11     9     5     0

Average turnaround time: 8.000000ms

Average waiting time: 4.333333ms

Process returned 0 (0x0)   execution time : 60.166 s
Press any key to continue.

```

LAB PROGRAM 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#include <string.h>
#define MAX 10
struct Process {
int pid, arrival, burst, completion, turnaround, waiting;
char type[10];
};
int main() {
int n;
struct Process SQ[MAX], UQ[MAX];
int sq = 0, uq = 0;
printf("Enter number of processes: ");
scanf("%d", &n);
for (int i = 0; i < n; i++) {
struct Process p;
printf("\nPID: "); scanf("%d", &p.pid);
printf("Arrival Time: "); scanf("%d", &p.arrival);
printf("Burst Time: "); scanf("%d", &p.burst);
printf("Type (System/User): "); scanf("%s", p.type);
if (strcmp(p.type, "System") == 0)
SQ[sq++] = p;
else
UQ[uq++] = p;
```

```

}
int current_time = 0;
int sq_front = 0, uq_front = 0;
struct Process result[MAX];
int r = 0;
while (sq_front < sq || uq_front < uq) {
while (sq_front < sq) {
struct Process *P = &SQ[sq_front];

if (current_time < P->arrival)
current_time = P->arrival;
current_time += P->burst;
P->completion = current_time;
P->turnaround = P->completion - P->arrival;
P->waiting = P->turnaround - P->burst;
result[r++] = *P;
sq_front++;
}
while (sq_front >= sq && uq_front < uq) {
struct Process *P = &UQ[uq_front];
if (current_time < P->arrival)
current_time = P->arrival;
current_time += P->burst;
P->completion = current_time;
P->turnaround = P->completion - P->arrival;
P->waiting = P->turnaround - P->burst;
result[r++] = *P;
uq_front++;
}
}
}

```

```

printf("\n%-5s %-8s %-8s %-8s %-12s %-12s %-8s\n",
"PID", "Type", "AT", "BT", "CT", "TAT", "WT");
printf("-----\n");
float total_tat = 0, total_wt = 0;
for (int i = 0; i < r; i++) {
printf("%-5d %-8s %-8d %-8d %-12d %-12d %-8d\n",
result[i].pid, result[i].type,
result[i].arrival, result[i].burst,
result[i].completion, result[i].turnaround, result[i].waiting);
total_tat += result[i].turnaround;
total_wt += result[i].waiting;
}
printf("-----\n");
printf("Average TAT : %.2f\n", total_tat / n);
printf("Average WT : %.2f\n", total_wt / n);
return 0;
}

```

```

Enter number of processes: 2

PID: 1
Arrival Time: 0
Burst Time: 4
Type (System/User): System

PID: 2
Arrival Time: 3
Burst Time: 6
Type (System/User): User

PID   Type   AT   BT   CT   TAT   WT
-----
1     System  0    4    4    4    0
2     User    3    6   10    7    1
-----

Average TAT : 5.50
Average WT  : 0.50

```

LAB PROGRAM 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic

```
#include <stdio.h>

#include <math.h>

#define MAX 10

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int find_lcm(int arr[], int n) {
    int res = arr[0];
    for (int i = 1; i < n; i++) {
        res = lcm(res, arr[i]);
    }
    return res;
}

int main() {
    printf("1WA24CS242\n");
    int n;
```



```

int C[MAX], P[MAX], A[MAX];
int remaining[MAX] = {0};
printf("Enter the number of processes: ");
scanf("%d", &n);
printf("Enter the CPU burst times:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &C[i]);
}

printf("Enter the time periods:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &P[i]);
}

printf("Enter arrival times:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &A[i]);
}

int hyper = find_lcm(P, n);
printf("LCM=%d\n\n", hyper);
float U = 0.0;
for (int i = 0; i < n; i++) {
    U += (float)C[i] / (float)P[i];
}

float bound;
bound = (float)n * (pow(2.0, 1.0 / (float)n) - 1.0);
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i + 1, C[i], P[i]);
}

```

```

printf("\n%f <= %f => %s\n", U, bound, (U <= bound) ? "true" : "false");
printf("Scheduling occurs for %d ms\n\n", hyper);
int last = -2;
for (int t = 0; t < hyper; t++) {
    for (int i = 0; i < n; i++) {
        if (t >= A[i] && (t - A[i]) % P[i] == 0) {
            remaining[i] = C[i];
        }
    }
    int sel = -1;
    int minP = 9999;
    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0 && P[i] < minP) {

            minP = P[i];
            sel = i;
        }
    }
    if (sel != last) {
        if (sel == -1)
            printf("%dms onwards: CPU is idle\n", t);
        else
            printf("%dms onwards: Process %d running\n", t, sel + 1);
        last = sel;
    }
    if (sel != -1)
        remaining[sel]--;
    }
    return 0;
}

```

```

1WA24CS242
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
Enter arrival times:
0 0
LCM=100

Rate Monotone Scheduling:
PID    Burst    Period
1       20       50
2       35       100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 24.646 s
Press any key to continue.

```

b) Earliest-deadline First

```

#include <stdio.h>

#define MAX 10

int main() {
    printf("1WA24CS242\n");

    int n;

    int burst[MAX], period[MAX], deadline[MAX];
    int remaining[MAX], abs_deadline[MAX];

    int time = 0;

    int hyper = 20;

    printf("Enter number of processes: ");

    scanf("%d", &n);

    printf("Enter Burst, Deadline, Period:\n");

    for (int i = 0; i < n; i++) {

        scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);

        remaining[i] = 0;

        abs_deadline[i] = deadline[i];
    }
}

```

```

}

printf("\nPID\tBurst\tDeadline\tPeriod\n");

for (int i = 0; i < n; i++) {

printf("%d\t%d\t%d\t%d\n", i + 1, burst[i], deadline[i], period[i]);

}

printf("\nScheduling occurs for %d ms\n\n", hyper);

while (time < hyper) {

for (int i = 0; i < n; i++) {

if (time % period[i] == 0) {

remaining[i] = burst[i];

abs_deadline[i] = time + deadline[i];

}

}

int selected = -1;

int earliest_deadline = 9999;

for (int i = 0; i < n; i++) {

if (remaining[i] > 0) {

if (abs_deadline[i] < earliest_deadline) {

earliest_deadline = abs_deadline[i];

}

}

}

selected = i;

}

}

if (selected == -1) {

printf("%dms : CPU is idle.\n", time);

} else {

```

```

printf("%dms : Task %d is running.\n", time, selected + 1);
remaining[selected]--;
}
time++;
}
return 0;
}

```

```

1WA24CS242
Enter number of processes: 3
Enter Burst, Deadline, Period:
2 4 5
3 7 20
2 8 10

PID      Burst  Deadline  Period
1         2       4          5
2         3       7         20
3         2       8         10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 1 is running.
8ms : Task 1 is running.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.

Process returned 0 (0x0)   execution time : 47.850 s
Press any key to continue.

```

c) Proportional Share Scheduling

```
#include<stdio.h>

#include <stdlib.h>
#include <time.h>

typedef struct
{
    char name[5]; int tickets; }
Process;

int main() {
    int n, total_tickets = 0; float total_T =0.0;
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
    { printf("\nProcess %d:\n", i + 1);
      sprintf(p[i].name, "P%d", i + 1);
      printf("Tickets: ");
      scanf("%d", &p[i].tickets);
      total_tickets += p[i].tickets;
      total_T +=p[i].tickets;
    }

    printf("\n--- Proportional Share Scheduling ---\n");
    printf("Enter the Time Period for scheduling: ");
    int m;

    scanf("%d",&m);
    for (int i = 0; i < m; i++)
    {
```

```

int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
    accumulated_tickets += p[j].tickets;
    if (winning_ticket <= accumulated_tickets)
    {
        winner_index = j; break;
    }
}

printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
}

for (int i = 0; i < n; i++)
{
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));
} return 0; }

```

```

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 21, Winner: P2
Tickets picked: 41, Winner: P3
Tickets picked: 7, Winner: P1
Tickets picked: 60, Winner: P3
Tickets picked: 48, Winner: P3

The Process: P1 gets 16.67% of Processor Time.
The Process: P2 gets 33.33% of Processor Time.
The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 23.494 s
Press any key to continue.
|

```

LAB PROGRAM 4

Write a C program to simulate:

a) Producer-Consumer problem using semaphores.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int mutex = 1;
```

```
int full = 0;
```

```
int empty = 5;
```

```
int x = 0;
```

```
void producer() {
```

```
    --mutex;
```

```
    ++full;
```

```
    --empty;
```

```
    x++;
```

```
    printf("\nProducer produces item %d", x);
```

```
    ++mutex;
```

```
}
```

```
void consumer() {
```

```
    --mutex;
```

```
    --full;
```

```
    ++empty;
```

```
    printf("\nConsumer consumes item %d", x);
```

```
    x--;
```

```
    ++mutex;
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    while (1) {
```

```
        printf("\n1. Producer\n2. Consumer\n3. Exit");
```

```
        printf("\nEnter choice: ");
```

```
        scanf("%d", &n);
```



```

switch (n) {
case 1:
    if ((mutex == 1) && (empty != 0))
        producer();
    else
        printf("Buffer is full!");
    break;

case 2:
    if ((mutex == 1) && (full != 0))
        consumer();
    else
        printf("Buffer is empty!");
    break;

case 3:
    exit(0);

default:
    printf("Invalid choice!");
}
}return 0; }

```

```

1. Producer
2. Consumer
3. Exit
Enter choice: 1

Producer produces item 1
1. Producer
2. Consumer
3. Exit
Enter choice: 1

Producer produces item 2
1. Producer
2. Consumer
3. Exit
Enter choice: 2

Consumer consumes item 2
1. Producer
2. Consumer
3. Exit
Enter choice: 2

Consumer consumes item 1
1. Producer
2. Consumer
3. Exit
Enter choice: 2
Buffer is empty!
1. Producer
2. Consumer
3. Exit
Enter choice: 3

Process returned 0 (0x0)   execution time : 120.546 s
Press any key to continue.
|

```

b) Dining-Philosopher's problem

```
#include <stdio.h>

#include <pthread>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;
    while(1)
    {
        printf(&quot;Philosopher %d is thinking.\n&quot;, id);
        sleep(1);
        if(id % 2 == 0)
        {
            pthread_mutex_lock(&f forks[left]);
            printf(&quot;Philosopher %d picked up left fork %d.\n&quot;,
                id, left);
            pthread_mutex_lock(&f forks[right]);
            printf(&quot;Philosopher %d picked up right fork %d.\n&quot;,
                id, right);
        }
        else
        {
            pthread_mutex_lock(&f forks[right]);
            printf(&quot;Philosopher %d picked up right fork %d.\n&quot;,
```

```

    id, right);
pthread_mutex_lock(&forks[left]);
printf(&quot;Philosopher %d picked up left fork %d.\n&quot;,
    id, left);
}
printf(&quot;Philosopher %d is eating.\n&quot;, id);
sleep(2);
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf(&quot;Philosopher %d put down forks %d and %d.\n\n&quot;,
    id, left, right);
}
return NULL;
}

int main()
{

    int i;
    int ids[N];
    for(i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for(i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i],
            NULL,
            philosopher,
            &ids[i]);
    }
}

```

```

}
for(i = 0; i < N; i++)
{
pthread_join(philosophers[i], NULL);
}
for(i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}
return 0;
}

```

```

Philosopher 2 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 1 put down forks 1 and 2.

Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 put down forks 3 and 4.

Philosopher 3 is thinking.
Philosopher 1 is thinking.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 2 put down forks 2 and 3.

Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 put down forks 0 and 1.

Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 4 put down forks 4 and 0.

Philosopher 4 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 1 put down forks 1 and 2.

Philosopher 1 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 picked up left fork 3.
Philosopher 2 picked up left fork 2.
Philosopher 3 is eating.
Philosopher 0 put down forks 0 and 1.

```

LAB PROGRAM 5

Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m;
```

```
    int i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], need[n][m];
```

```
    int avail[m];
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < m; j++)
```

```
        {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        }
```

```
}
```

```
printf("Enter Maximum Demand Matrix:\n");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    for(j = 0; j < m; j++)
```

```
    {
```

```
        scanf("%d", &max[i][j]);
```

```
    }
```

```
}
```

```
printf("Enter Available Resources:\n");
```

```
for(i = 0; i < m; i++)
```

```
{
```

```
    scanf("%d", &avail[i]);
```

```
}
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    for(j = 0; j < m; j++)
```

```
    {
```

```
        need[i][j] = max[i][j] - alloc[i][j];
```

```
    }
```

```
}
```

```
int finish[n];
```

```
int safeSeq[n];
```

```
int work[m];
```

```
for(i = 0; i < n; i++)
```

```
{  
    finish[i] = 0;  
}
```

```
for(i = 0; i < m; i++)  
{  
    work[i] = avail[i];  
}
```

```
int count = 0;
```

```
while(count < n)  
{  
    int found = 0;
```

```
    for(i = 0; i < n; i++)  
    {  
        if(finish[i] == 0)  
        {  
            for(j = 0; j < m; j++)  
            {  
                if(need[i][j] > work[j])  
                    break;  
            }  
        }
```

```
        if(j == m)  
        {  
            for(k = 0; k < m; k++)  
            {  
                work[k] += alloc[i][k];
```

```

    }

    safeSeq[count++] = i;
    finish[i] = 1;
    found = 1;
}
}
}

if(found == 0)
{
    break;
}
}

if(count == n)
{
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSeq[i]);

        if(i != n - 1)
            printf(" -> ");
    }
}
else
{

```



```

        printf("\nSystem is NOT in a safe state.\n");
    }

    return 0;
}
}

```

```

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
 2 0 0
3 0 3
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0)  execution time : 65.397 s
Press any key to continue.
|

```

2) Deadlock Detection

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m;
```

```
    int i, j, k;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
printf("Enter the number of resources: ");  
scanf("%d", &m);
```

```
int alloc[n][m], request[n][m];  
int avail[m];
```

```
printf("Enter the allocation matrix:\n");  
for(i = 0; i < n; i++)  
{  
    printf("Process %d: ", i);  
    for(j = 0; j < m; j++)  
    {  
        scanf("%d", &alloc[i][j]);  
    }  
}
```

```
printf("Enter the request matrix:\n");  
for(i = 0; i < n; i++)  
{  
    printf("Process %d: ", i);  
    for(j = 0; j < m; j++)  
    {  
        scanf("%d", &request[i][j]);  
    }  
}
```

```
printf("Enter the available resources: ");  
for(i = 0; i < m; i++)  
{
```

```
    scanf("%d", &avail[i]);  
}
```

```
int finish[n], safeSeq[n];  
int work[m];
```

```
for(i = 0; i < m; i++)  
{  
    work[i] = avail[i];  
}
```

```
for(i = 0; i < n; i++)  
{  
    finish[i] = 0;  
}
```

```
int count = 0;
```

```
while(count < n)  
{  
    int found = 0;
```

```
    for(i = 0; i < n; i++)  
    {  
        if(finish[i] == 0)  
        {  
            for(j = 0; j < m; j++)  
            {  
                if(request[i][j] > work[j])  
                    break;
```

```

    }

    if(j == m)
    {
        for(k = 0; k < m; k++)
        {
            work[k] += alloc[i][k];
        }

        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
    break;
}

if(count == n)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d ", safeSeq[i]);
    }
}

```

```

else
{
    printf("\nDeadlock detected!\n");
    printf("Processes in deadlock: ");

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
            printf("P%d ", i);
    }
}

return 0;
}

```

```

Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1
Process returned 0 (0x0)   execution time : 52.084 s
Press any key to continue.
|

```

LAB PROGRAM 6

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit

b) Best-fit

c) First-fit

```
#include <stdio.h>
```

```
#define MAX 25
```

```
void firstFit(int blockSizes[], int n, int processSizes[], int m)
{
    int allocation[MAX];

    for(int i = 0; i < m; i++)
        allocation[i] = -1;

    for(int i = 0; i < m; i++)
    {
        for(int j = 0; j < n; j++)
        {
            if(blockSizes[j] >= processSizes[i])
            {
                allocation[i] = j + 1;
                blockSizes[j] = 0;
                break;
            }
        }
    }

    printf("\n--- First Fit ---\n");
    printf("%-15s %-15s %s\n", "Process No.", "Process Size", "Block No.");

    for(int i = 0; i < m; i++)
    {
        printf("%-15d %-15d", i + 1, processSizes[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i]);
    }
}
```

```

        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSizes[], int n, int processSizes[], int m)
{
    int allocation[MAX];

    for(int i = 0; i < m; i++)
        allocation[i] = -1;

    for(int i = 0; i < m; i++)
    {
        int bestBlock = -1;

        for(int j = 0; j < n; j++)
        {
            if(blockSizes[j] >= processSizes[i])
            {
                if(bestBlock == -1 || blockSizes[j] < blockSizes[bestBlock])
                    bestBlock = j;
            }
        }

        if(bestBlock != -1)
        {
            allocation[i] = bestBlock + 1;
            blockSizes[bestBlock] = 0;
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("%-15s %-15s %s\n", "Process No.", "Process Size", "Block No.");

    for(int i = 0; i < m; i++)
    {
        printf("%-15d %-15d", i + 1, processSizes[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
    }
}

```

```
}
```

```
void worstFit(int blockSizes[], int n, int processSizes[], int m)
{
    int allocation[MAX];

    for(int i = 0; i < m; i++)
        allocation[i] = -1;

    for(int i = 0; i < m; i++)
    {
        int worstBlock = -1;

        for(int j = 0; j < n; j++)
        {
            if(blockSizes[j] >= processSizes[i])
            {
                if(worstBlock == -1 || blockSizes[j] > blockSizes[worstBlock])
                    worstBlock = j;
            }
        }

        if(worstBlock != -1)
        {
            allocation[i] = worstBlock + 1;
            blockSizes[worstBlock] = 0;
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("%-15s %-15s %s\n", "Process No.", "Process Size", "Block No.");

    for(int i = 0; i < m; i++)
    {
        printf("%-15d %-15d", i + 1, processSizes[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i]);
        else
            printf("Not Allocated\n");
    }
}
```

```
int main()
```



```

{
    int n, m;
    int blockSizes1[MAX], blockSizes2[MAX], blockSizes3[MAX];
    int originalBlocks[MAX];
    int processSizes[MAX];

    printf("Enter number of memory blocks: ");
    scanf("%d", &n);

    printf("Enter sizes of %d memory blocks:\n", n);

    for(int i = 0; i < n; i++)
    {
        scanf("%d", &originalBlocks[i]);
        blockSizes1[i] = originalBlocks[i];
        blockSizes2[i] = originalBlocks[i];
        blockSizes3[i] = originalBlocks[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &m);

    printf("Enter sizes of %d processes:\n", m);

    for(int i = 0; i < m; i++)
        scanf("%d", &processSizes[i]);

    firstFit(blockSizes1, n, processSizes, m);
    bestFit(blockSizes2, n, processSizes, m);
    worstFit(blockSizes3, n, processSizes, m);

    return 0;
}

```

```

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.   Process Size   Block No.
1             212           2
2             417           5
3             112           3
4             426          Not Allocated

--- Best Fit ---
Process No.   Process Size   Block No.
1             212           4
2             417           2
3             112           3
4             426           5

--- Worst Fit ---
Process No.   Process Size   Block No.
1             212           5
2             417           2
3             112           4

```

LAB PROGRAM 7

Write a C program to simulate page replacement algorithms.

- FIFO
- LRU
- Optimal

```

#include <stdio.h>
int search(int frames[], int n, int page) {
    for (int i = 0; i < n; i++)
        if (frames[i] == page) return 1;
    return 0;
}
void print_frames(int frames[], int n) {
    printf("[");
    for (int i = 0; i < n; i++) {
        if (frames[i] == -1) break;

```

```

if (i > 0) printf(" ");
printf("%d", frames[i]);
}
printf("]");
}

void fifo(int pages[], int num_pages, int num_frames) {
int frames[num_frames];
for (int i = 0; i < num_frames; i++) frames[i] = -1;
int page_faults = 0, next = 0;
printf("\n--- FIFO Page Replacement ---\n");
for (int i = 0; i < num_pages; i++) {
printf("Page %d -> ", pages[i]);
if (!search(frames, num_frames, pages[i])) {
frames[next] = pages[i];
next = (next + 1) % num_frames;
page_faults++;
}
print_frames(frames, num_frames);
printf("\n");
}
printf("Total Page Faults (FIFO): %d\n", page_faults);
}

void lru(int pages[], int num_pages, int num_frames) {
int frames[num_frames];
int last_used[num_frames];
for (int i = 0; i < num_frames; i++) { frames[i] = -1; last_used[i] = -1; }

int page_faults = 0;
printf("\n--- LRU Page Replacement ---\n");
for (int i = 0; i < num_pages; i++) {
printf("Page %d -> ", pages[i]);
int found = 0;
for (int j = 0; j < num_frames; j++) {
if (frames[j] == pages[i]) {
last_used[j] = i;
found = 1;
break;
}
}
if (!found) {
int pos = -1;
for (int j = 0; j < num_frames; j++) {
if (frames[j] == -1) { pos = j; break; }
}
}
}

```

```

if (pos == -1) {
    int lru_time = last_used[0], lru_idx = 0;
    for (int j = 1; j < num_frames; j++) {
        if (last_used[j] < lru_time) { lru_time = last_used[j]; lru_idx = j; }
    }
    pos = lru_idx;
}
frames[pos] = pages[i];
last_used[pos] = i;
page_faults++;
}
print_frames(frames, num_frames);
printf("\n");
}
printf("Total Page Faults (LRU): %d\n", page_faults);
}

void optimal(int pages[], int num_pages, int num_frames) {
    int frames[num_frames];
    for (int i = 0; i < num_frames; i++) frames[i] = -1;
    int page_faults = 0;
    printf("\n--- Optimal Page Replacement ---\n");
    for (int i = 0; i < num_pages; i++) {
        printf("Page %d -> ", pages[i]);
        if (!search(frames, num_frames, pages[i])) {
            int pos = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) { pos = j; break; }
            }
            if (pos == -1) {
                int farthest = -1, replace = -1;
                for (int j = 0; j < num_frames; j++) {
                    int next_use = num_pages;
                    for (int k = i + 1; k < num_pages; k++) {
                        if (pages[k] == frames[j]) { next_use = k; break; }
                    }
                    if (next_use > farthest) { farthest = next_use; replace = j; }
                }
                pos = replace;
            }
            frames[pos] = pages[i];
            page_faults++;
        }
        print_frames(frames, num_frames);
    }
}

```

```

printf("\n");
}
printf("Total Page Faults (Optimal): %d\n", page_faults);
}

int main() {
int num_pages, num_frames;
printf("Enter number of pages: ");
scanf("%d", &num_pages);
int pages[num_pages];
printf("Enter page reference string:\n");
for (int i = 0; i < num_pages; i++)
scanf("%d", &pages[i]);
printf("Enter number of frames: ");
scanf("%d", &num_frames);
fifo(pages, num_pages, num_frames);
lru(pages, num_pages, num_frames);
optimal(pages, num_pages, num_frames);
return 0;
}

```

```

Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

```

```

--- FIFO Page Replacement ---

```

```

Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

```

```

--- LRU Page Replacement ---

```

```

Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]
Total Page Faults (LRU): 9

```

```

--- Optimal Page Replacement ---

```

```

Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]
Total Page Faults (Optimal): 7

```

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.